Introduction to MCMCs Module 1: Critical properties of the Ising model

M. Barbieri * **M. Dell'Anna** *

* *University of Pisa, Largo Pontecorvo 3, I-56127 Pisa, Italy*
(email: m.barbieri20@studenti.unipi.it, m.dellanna2@studenti.unipi.it)

Abstract: The aim of this report is to verify some critical properties of the Euclidean $D = 2$ Ising model through Monte Carlo simulations. We focus on critical temperatures and critical exponents γ, ν, and z, which determine all the others – via scaling relations. The simulations are performed on square, triangular, and hexagonal lattices, with toroidal boundary conditions. Data from Wolff and Metropolis algorithms are compared.

1. INTRODUCTION

The Ising model is the simplest statistical model for a ferromagnetic system. It consists of a lattice of spins σ_i, which can take only value $+1$ and -1, and the Hamiltonian has form

$$H = -J \sum_{\langle ij \rangle} \sigma_i \sigma_j - \sum_i h_i \sigma_i, \tag{1}$$

where by $\langle ij \rangle$ we mean that the sum only runs on adjacent sites, and h_i is an external background magnetic field. The energy manifestly has a $\mathbb{Z}_2$ symmetry.

The dynamics of said system, in two or more dimensions, spontaneously breaks this symmetry at non-vanishing temperature, leading to a phase transition. In two dimensions, the critical behavior of the Ising model was first fully described by L. Onsager.

In this report, we verify the theoretical results through numerical simulations. The discussion is organized as follows:

- in Section 2 we briefly summarize the procedures, choices, and conventions we used for the simulations;
- in Section 3 we present the statistical and fitting methods that have been used to analyze the numerical simulations;
- in Section 4 we show and briefly comment on the obtained results.

2. METHODS OF INVESTIGATION

2.1 Units

Throughout the document, we set the Boltzmann constant $k_B = 1$ and we work in energy units of J, therefore $J = 1$.

2.2 Simulation algorithms

In order to investigate critical properties of the model, we rely on Monte Carlo simulation of the observables, with the underlying distribution being the usual Boltzmann distribution:

$$P(\sigma_1, ...\sigma_n) = \frac{1}{Z} \cdot \exp[-\beta H(\sigma_1, ..., \sigma_n)]. \tag{2}$$

The only observables that we need for our purposes are the average magnetization per unit volume m, its absolute value $|m|$ and its square power m^2. Two different Markov Chains are built to sample their distribution:

The Metropolis MC with deterministic choice of sites to be updated; for the associated Markov Chain to be aperiodic, it is necessary that at each step an extraction is made in each spin of the lattice. This procedure is highly parallelizable; therefore, we implemented it via the GPU (Apple Silicon M1 Pro, in our case). This program is written in `C++` and `Metal`[1].

To avoid data races, we first update sites with even abscissa and even ordinate, then those with even abscissa and odd ordinate, etc.... This works straightforwardly with the three geometries.

The associated Markov Chain is manifestly irreducible and aperiodic. However, there is a subtle point about the balance condition: in fact, during each of the four moments in a step, the Markov Chain associated to the transformation is not irreducible (e.g. during the first moment, it does not update spins with odd abscissa and ordinate). Nevertheless, the balance condition is satisfied, as each of the four sub-chains samples the conditioned probability.

— Proof —

Let us consider a system composed of two subsystems A and B. Let $\mathcal{H}_A$ and $\mathcal{H}_B$ the free vector spaces generated by the states of A and B, respectively, and let $\pi_{a,b}$ (with $a \in A, b \in B$) the p.d.f. to be sampled.

[1] For an introduction on how to use Metal for scientific computing on Apple Silicon processors, see [1].
The repo attached to that paper does not free the allocated Metal objects, and this for long simulations can lead to large amounts of allocated RAM. Because of the way Metal handles memory, this must be done via `object->release()` and not via `delete object`. Moreover, this resource does not describe asynchronous communication between CPU and GPU, which could be a relevant feature in a performance sensitivity context.

The analogous of the sub-chains that update sites of given parity of abscissa and ordinate, in this case, is a pair of irreducible (on the subsystem) and aperiodic Markov Chain $\mathcal{M}_1, \mathcal{M}_2$ that act only on the first or on the second subsystem, in the same order, but that have as asymptotic probability density function the conditioned probability $\pi_{i|j}$ and $\pi_{j|i}$. Let W_i be the stochastic matrix associated with $\mathcal{M}_i$. We can state the balance condition as

$$\begin{aligned} \forall b_0 \in B, \quad & W_1 \sum_a \pi_{a|b_0} |a\rangle \otimes |b_0\rangle = \sum_a \pi_{a|b_0} |a\rangle \otimes |b_0\rangle, \\ \forall a_0 \in A, \quad & W_2 \sum_b \pi_{b|a_0} |a_0\rangle \otimes |b\rangle = \sum_b \pi_{b|a_0} |a_0\rangle \otimes |b\rangle. \end{aligned} \tag{3}$$

Using Bayes theorem, it is possible to show that

$$\begin{aligned} |\pi\rangle &= \sum_{a,b} \pi_{a,b} |a\rangle \otimes |b\rangle \\ &= \sum_{b_0} \pi_{b_0} \sum_a \pi_{a|b_0} |a\rangle \otimes |b_0\rangle \\ &= \sum_{a_0} \pi_{a_0} \sum_b \pi_{b|a_0} |a_0\rangle \otimes |b\rangle \end{aligned} \tag{4}$$

and so is an eigenvector with unitary eigenvalue of W_1 and W_2, so it is also an eigenvector of unitary eigenvalue of $W = W_1 \cdot W_2$, and that concludes the proof.

The generalization in the case of four sub-systems (that is our case) is trivial.

Wolff's MC a single-cluster algorithm that is hard to parallelize but much more efficient than Metropolis, especially close to the critical point. This program is written in plain `C`.

As far as the random number generator is concerned, we use PCG32 [2], which is high-quality and high-speed.

We use AoS memory management. This proves to be better not only for code readability, but even more efficient as it allows us to identify neighboring sites via pointers[2].

Both simulation algorithms accept as arguments a `u.p.s.`–updates per sample parameter that allows only one data item to be saved every `u.p.s.` steps of the simulation.

All codes are available at our repository on GitHub [3].

2.3 Lattices

Simulations are carried out on two-dimensional lattices with periodic, untwisted boundary conditions.

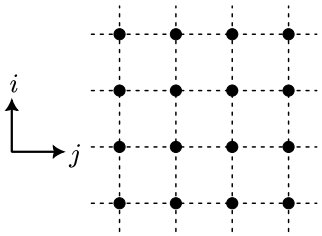

Fig. 1. Lattice with square geometry.

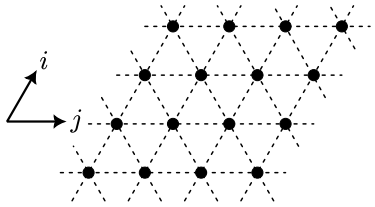

Fig. 2. Lattice with triangular geometry.

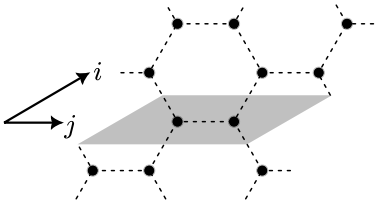

Fig. 3. Lattice with hexagonal geometry. A unit cell is shadowed.

In order both to compare critical temperatures with expected temperatures and to verify the *universality* of critical exponents, we adopt three lattice families:

- *square geometry*: with $(i; j)$ Cartesian coordinates, lattice of $L \times L$ points;
- *triangular geometry*: with $(i; j)$ non-orthogonal Cartesian coordinates, and a lattice of $L \times L$ spins;
- *hexagonal geometry*: with coordinates $(i; j; k)$ and a lattice $L \times L \times 2$; the compact coordinate k discides the spin within the unit cell.

A visual representation of the three is shown in Fig. 1, Fig. 2 and Fig. 3.
In both cases, we save spins in RAM in row-major order:

```
lattice[i,j] := lattice[i*L+j]
```

for square and triangular lattices and

```
lattice[i,j,k] := lattice[i*2L+2j+k]
```

for the hexagonal lattice.

Each spin in the lattice is represented with a `struct`

```
struct cell {
 int64_t      value;
 struct cell* neighbors[N_NEIGH];
};
```

containing the spin value and neighbors. Important performance increase is obtained using pointers to neighbors instead of indices with the Wolff algorithm[3], but the opposite in the case of Metropolis.

To avoid unnecessary "if-checks" in functions, we use macros and compile three versions of each program, one per lattice geometry.

3. DATA ANALYSIS

This section attempts to analyze the subtleties we found mainly in the treatment of uncertainties and finite size effects.

In the Ising model, the order parameter is the expectation value of the magnetization

[2] Attention must be paid to some subtleties regarding how these addresses are created. For example, code running on the GPU sees different virtual addresses than on the CPU.

[3] By using indices instead of pointers, the run time worsens by a factor of 1.5.

$$\langle m \rangle = \langle \sigma_i \rangle \tag{5}$$

(m does not depend on i because of translation invariance).

The S.S.B. of the critical (infinite) system manifests itself in the finite system behavior with the emergence of two marked peaks into the probability density function of the average magnetization, at low temperatures. At higher L, the two peaks become sharper, as shown in figure Fig. 13.

3.1 Critical temperature and exponents

We now introduce the reduced magnetic susceptibility, which is the core of our investigation, defined as follows:

$$\chi = \beta V \left[\langle m^2 \rangle - \langle |m| \rangle^2 \right] = \beta V \sigma^2_{|m|}, \tag{6}$$

where $\beta = 1/T$ is the inverse temperature.

In the thermodynamic limit, χ has a point of non-analyticity at the critical temperature, which allows to define the critical exponent[4] γ:

$$\chi \sim |\beta - \beta_{\text{cr}}|^{-\gamma}. \tag{7}$$

In a finite lattice, clearly such a point of non-analyticity does not exist. Therefore, finite size effects must be taken into account in order to calculate the critical temperature. Critical exponents γ and ν can be found by looking at the scaling laws related to the magnetic susceptibility; in particular, calling $\chi^{(L)}(\beta)$ the susceptibility associated to a given lattice of size L, must follow:

$$\chi^{(L)}(\beta) = L^{\frac{\gamma}{\nu}} \cdot f\left(L^{\frac{1}{\nu}} \cdot (\beta - \beta_{\text{cr}}) \right) \cdot \left[1 + O\left(\frac{1}{L} \right) \right] \tag{8}$$

where f is called *universal function*. Furthermore, in our case the universal function is unimodal, therefore peak coordinates $\left(\beta^{(L)}_{\text{max}}; \chi^{(L)}_{\text{max}} \right)$ of susceptibility satisfy the scaling relations:

$$\begin{cases} \beta^{(L)}_{\text{max}} \simeq \beta_{\text{cr}} + b \cdot L^{-\frac{1}{\nu}} \\ \chi^{(L)}_{\text{max}} \simeq c_0 + c_1 \cdot L^{\frac{\gamma}{\nu}} \end{cases} \tag{9}$$

Then we estimate the values of critical temperature and exponents by the following procedure:

1. chosen the lattice side L, sample the statistical distribution at various βs;
2. for each sample, we compute the magnetic susceptibility, and we identify its peak coordinates via a quadratic fit, as shown in Fig. 4;
3. we repeat 1. and 2. varying L;
4. we extract critical temperature and exponents via an exponential fit, using (9), as shown in Fig. 11 and Fig. 12.

To verify stability of fit in 2., we remove some points, then compare the results between the complete and reduced fit by the quantity $b = \left(\beta^{(L)}_{\text{max}} - \beta_{\text{red}} \right) / \sigma_\beta$. We show in Fig. 10 histograms of b, that confirm fit stability.

We consider the following systematic errors:

- the pseudo-random number generator generates 32-bit integers. We expect this to shift the simulation temperatures by about 2^{-32}, therefore is completely negligible.
- To find the maximum of χ_L, we sample it around its maximum and perform a fit with a parabolic model; systematic errors arise from neglected higher-power contributions.

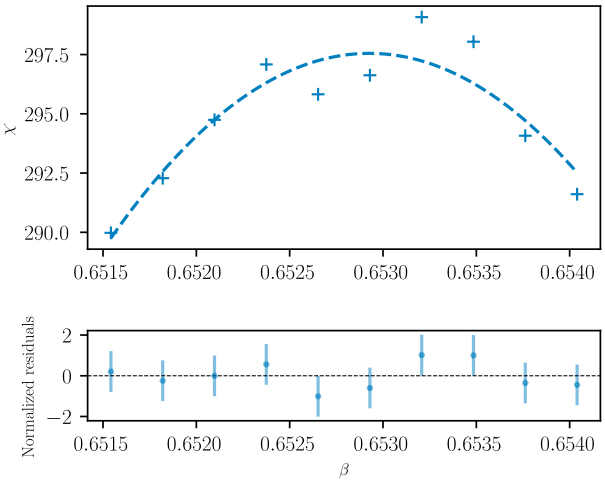

Fig. 4. Example of quadratic fit for the maximum of the susceptibility.
This was performed at $L = 80$ in an hexagonal lattice.

In order to neglect the latter, we tune simulation parameters so that the statistical error is about 10 times larger than the systematic one (and so the uncertainties can be estimated via a standard statistical data analysis), proceeding as follows:

1. we perform rough preliminary measurements to estimate $\beta^{(L)}_{\text{max}}, \chi^{(L)}_{\text{max}}$, m and m^2 sample variances, and the integrated correlation time, as shown in Fig. 7, Fig. 8, and Fig. 9;
2. we expect that by expanding $\chi_{L(\beta)}$ around the maximum,

$$\chi^{(L)}(\beta) = \chi^{(L)}_{\text{max}} - \frac{1}{2} \left(\frac{\beta - \beta^{(L)}_{\text{max}}}{\beta^{(L)}_0} \right)^2 + \frac{1}{6} \chi^{(L)}_3 \left(\frac{\beta - \beta^{(L)}_{\text{max}}}{\beta^{(L)}_0} \right)^3 + \ldots, \tag{10}$$

   where $\chi^{(L)}_3$ is a numerical factor in the order of unity;
3. we make new simulations in which we tune the parameters to have statistical uncertainty about ten times larger than the cubic term, but ten times smaller than the quadratic one, so that the signal-to-noise ratio is sufficiently high. We estimate the correct number of samples from preliminary sample variance, the `u.p.s.` from the integrated correlation time, and temperature range from $\beta^{(L)}_{\text{max}}, \chi^{(L)}_{\text{max}}$:

[4] *A priori*, the limits for $\beta \to \beta^{\pm}_{\text{cr}}$ could lead to different γs. This is not the case for the Ising model.

$$\begin{cases} \sigma^2 \simeq \dfrac{9}{2 \cdot 10^4} \left(\chi^{(L)}_{\max}\right)^{-1} \\ \Delta\beta^{(L)} = \sqrt{\dfrac{3}{5\sqrt{2}}}\, \beta_0^{(L)} \left(\chi^{(L)}_{\max}\right)^{1/4}) \\ \qquad\quad \approx 0.65\, \beta_0^{(L)} \left(\chi^{(L)}_{\max}\right)^{1/4} \end{cases} \tag{11}$$

In our case, we sample 10–15 points in that interval, as shown in Fig. 4.

3.2 Statistical uncertainties of primary observables

With regard to the primary observables, we make two important remarks. Firstly, two data generated a few steps apart are highly correlated, then we need to be cautious when treating statistical uncertainties. Secondly, it is desirable to save as little data as possible, for it takes up time and memory storage to do so. Therefore, we must save data once every `u.p.s.` steps of the simulation (an integer parameter), so as to solve both of these problems. In order to find the right value for this parameter, we need to cope with two quantities, the exponential autocorrelation time τ_{exp} and the integrated exponential time τ_{int}.

Estimate of τ_{exp} In the Markov Chains Monte Carlo formalism,

$$\tau_{\text{exp}} = -\ln(\Lambda), \tag{12}$$

where Λ is the largest modulus eigenvalue other than 1 in the spectre of the stochastic matrix W. We stress that the exponential autocorrelation time is intrinsically related to the Markov chain, as for any observable O, letting $C_O(k)$ be the autocorrelation relative to O at lag k, it is true that, for almost every initial vector in the free vector space of states:

$$C_O(k) \overset{k\to\infty}{\sim} \exp\left[-\frac{k}{\tau_{\text{exp}}}\right] \tag{13}$$

Furthermore, we point out that the autocorrelation time follows a scaling law, close to critical temperature. That is known as critical slowing down:

$$\tau_{\text{exp}} \sim L^z \tag{14}$$

that defines a new critical exponent z. In order to characterize it, after finding the critical temperature as explained in Section 3.1, we perform simulations at β_{cr} for different Ls and we extract τ_{exp} via an exponential fit, as shown in Fig. 5; finally, we estimate z by means of another exponential fit, as shown in Fig. 14 and Fig. 15. There are however some subtleties to be taken into account for the first exponential fit:

- autocorrelation gets eventually drowned in noise, hence high lags are to be excluded for the fit;
- for some simulations (e.g. Wolff on small lattices) this happens after very few iterations;
- small lags do not present the exponential decay of autocorrelation, due to other stochastic matrix eigenvectors' contributions.

We empirically found the range for autocorrelation values $[0.1, 0.4]$ to give (on average) reasonable results: enough points to perform the fit, decent exponential decay and non-noisy curve. The result is good enough for the purposes of this investigation, even being rough.

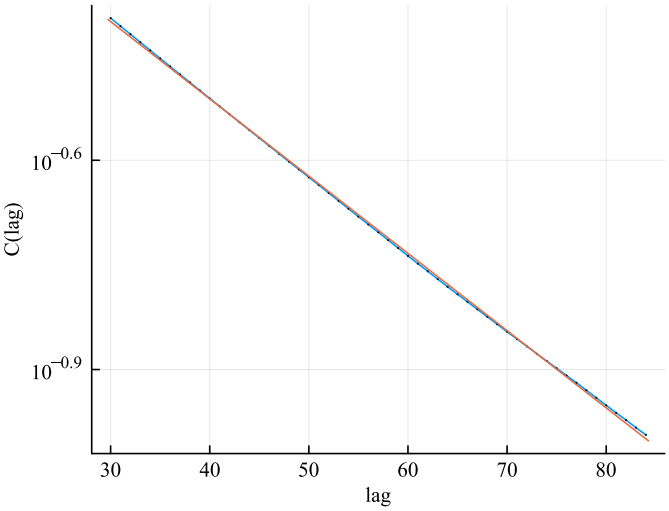

Fig. 5. Exponential decay of autocorrelation; orange curve is the best fit exponential curve.

Estimate of τ_{int} This second autocorrelation time is deeply related to the variance of the sample average, and it depends on the particular observable taken into account. In fact, given the observable O with σ^2_O variance, let $\overline{O} = \frac{1}{N}\sum_{i=1}^{N} O_i$ be the sample mean. It can be shown that the variance of this random variable is:

$$\sigma^2_{\overline{O}} = \frac{\sigma^2_O}{N^2} \cdot \sum_{i=1}^{N} \sum_{j-i} C_O(|i-j|) \overset{(*)}{\simeq} \frac{\sigma^2_O}{N}\left(1 + 2\tau^O_{\text{int}}\right), \tag{15}$$

where we defined:

$$\tau^O_{\text{int}} = \sum_{k=1}^{+\infty} C_O(k). \tag{16}$$

We point out that in (15).$(*)$ we extend the series up to infinity as following contribution are exponentially damped by τ_{exp}. For us to be able to identify the integrated autocorrelation time then, we follow a *blocking* procedure: starting from the original sample $O_1; ...; O_N$ we build a reduced one where

$$O^k_i = \frac{1}{k} \sum_{j=k\cdot i+1}^{k\cdot(i+1)} O_i \tag{17}$$

and compute the sample variance. As we increase k, the measures in the reduced sample get more and more uncorrelated, until the sample variance of the mean saturates to $\sigma^2_{\overline{O}}$, as shown in Fig. 6. We implemented an algorithm that estimates the smallest size $k^\star$ for which the saturation occurs.

Finally, we compute the sample variance σ^2_F and use (15) to find τ^O_{int}. At critical temperature, the integrated autocorrelation time of the observable $|m|$ follows a scaling law, as well:

$$\tau^{|m|}_{\text{int}} \sim L^{z'}. \tag{18}$$

We compute this exponent via an exponential fit, shown in Fig. 14 and Fig. 15.

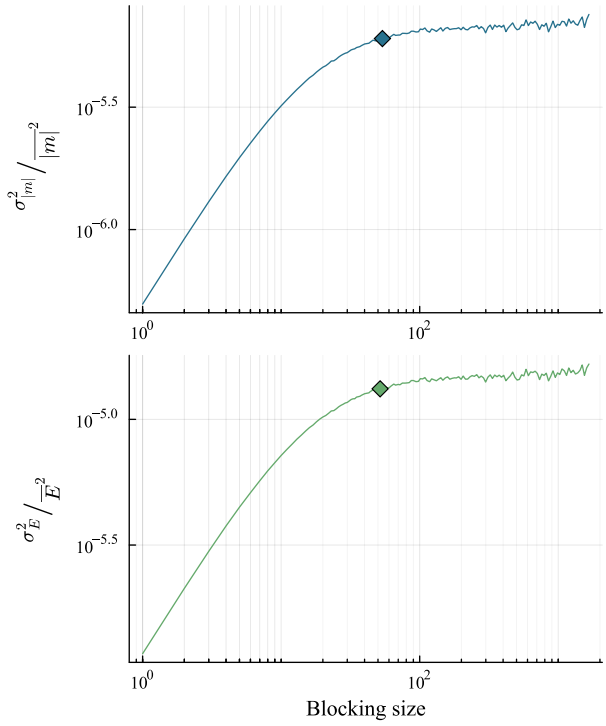

Fig. 6. Value of the sample variance of the blocked sample, varying the block size, for the average magnetization and energy. Diamond marks the $k^\star$.

3.3 Statistical uncertainties of secondary observables

For the purposes of this report, we only need reliable estimates of the uncertainties for the magnetic susceptibility, as it serves to fit the critical inverse temperature β_{cr} and the critical exponents γ and ν. Being a secondary observables (it depends on both m^2 and $|m|$), we use a Jackknife method: using the blocked data, we start from N measures of our primary observables and generate N leave-one-out mock samples; one can prove that:

$$\sigma_{\overline{O}} \simeq N \cdot \left(\overline{O_J^2} - \overline{O_J}^2 \right) \tag{19}$$

with $\overline{O}_J$ being the mean of the mock samples.

The two procedures (blocking and Jackknife) give robust statistical meaning to the uncertainties of the magnetic susceptibility, and guarantee that it is sufficient to launch longer simulations in order to get an arbitrary precision.

4. RESULTS

We finally present the results obtained with different geometries and algorithms. All experimental measurements are available at Tab. 1.

Thanks to the attention paid in the choice of the sampling intervals of β and to the number of measurements, most of the quadratic fits performed to find the peak of χ show a χ^2/d.o.f compatible to 1 and all of them compatible within 2σ. We recall that the fit for τ_{exp} was done for operational needs (less data, uncorrelated measures), and the error associated with that result has no statistical meaning.

An interesting conclusion we can extract from this work is the comparison between the Metropolis and Wolff algorithms; both of them are able to simulate the Ising model, giving results that are well in agreement with the theory. Nevertheless, as far as efficiency is concerned, Metropolis shows a much heavier critical slowing down close to critical point, and in general greater correlation times, even though the steps are whole lattice updates and `GPU`'s raw computing power is being used.

ACKNOWLEDGEMENTS

https://github.com/mbar02/nummet-public/raw/refs/heads/master/report/Module%201/pics/2mhh3d0hx1gb1.png

REFERENCES

1. Gebraad L, Fichtner A (2023) Seamless GPU Acceleration for C++-Based Physics with the Metal Shading Language on Apple's M Series Unified Chips. Seismological Research Letters. https://doi.org/10.1785/0220220241
2. O'Neill ME (2014) PCG: A Family of Simple Fast Space-Efficient Statistically Good Algorithms for Random Number Generation. Claremont, CA
3. Our GitHub repository. https://github.com/mbar02/nummet-public

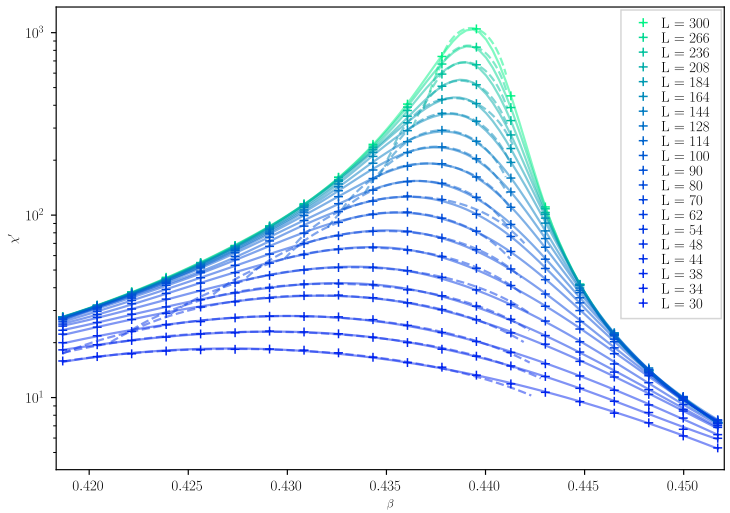

Fig. 7. Preliminary measurement of the reduced susceptibility of the square lattice.

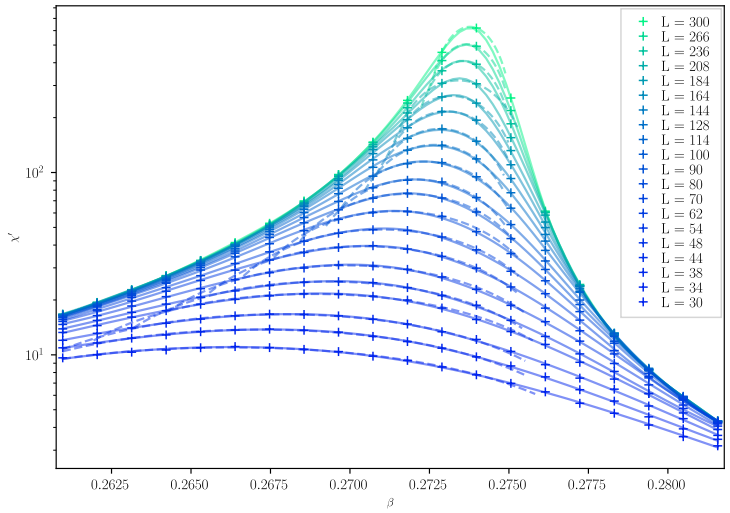

Fig. 8. Preliminary measurement of the reduced susceptibility of the triangular lattice.

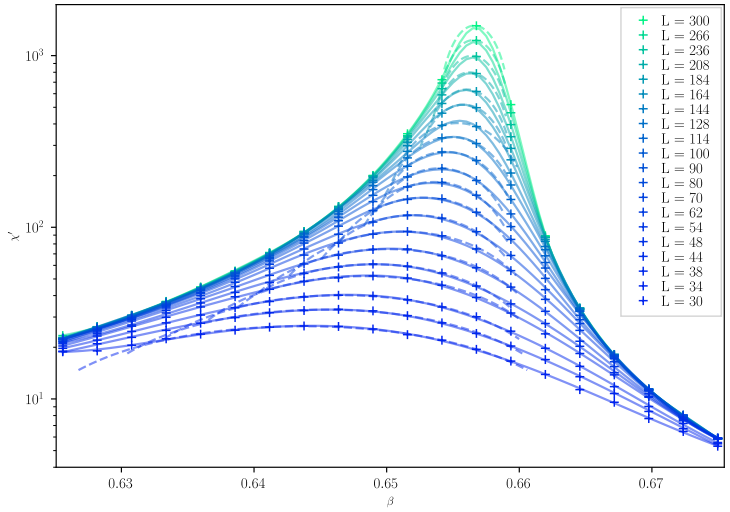

Fig. 9. Preliminary measurement of the reduced susceptibility of the hexagonal lattice.

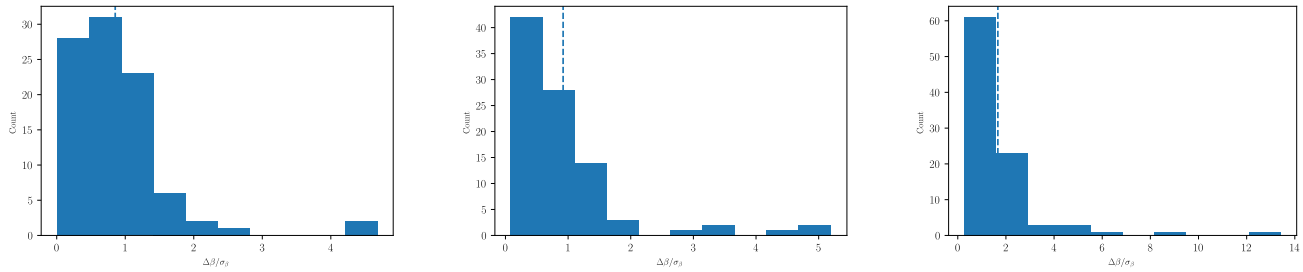

Fig. 10. Stability check of the $\beta_{\max}^{L}$ fit. Histograms represent the counts of the parameter $b = \frac{\Delta\beta}{\sigma_\beta}$ when half the points are removed (left), average of b removing one point at a time (middle), max of b removing one point at a time (right). Dashed line is the mean.

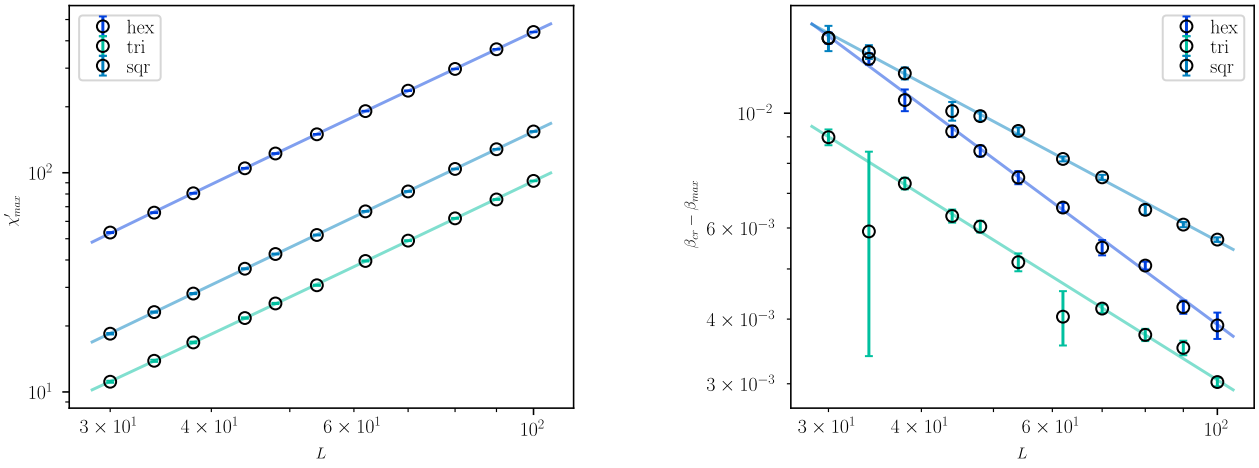

Fig. 11. Scaling of $\beta_{\max}$ and $\chi'_{\max}$ with Metropolis algorithm.

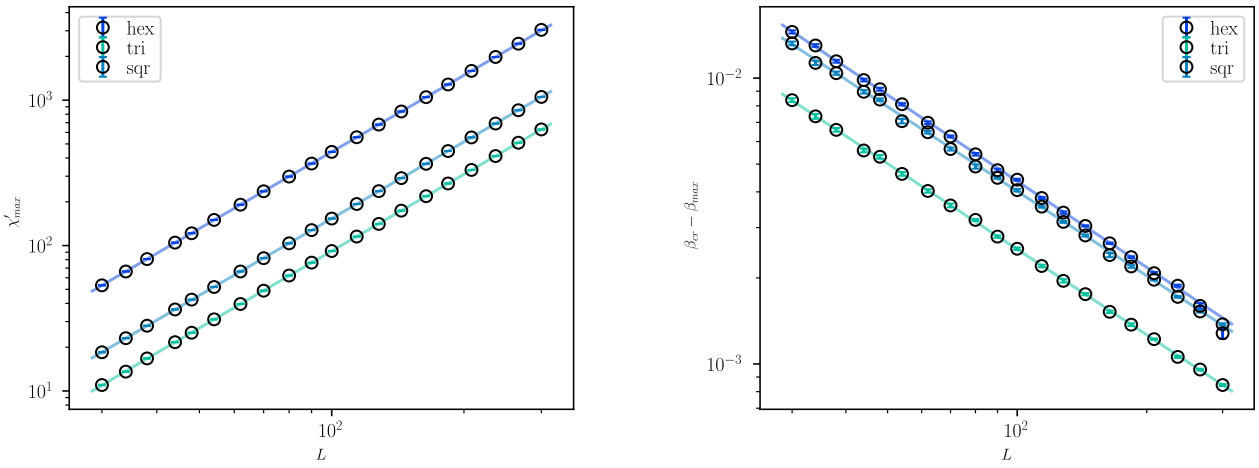

Fig. 12. Scaling of $\beta_{\max}$ and $\chi'_{\max}$ with Wolff algorithm.

Geometry	β_{cr}	γ	ν	z	z'
Square (Wolff)	0.44074(4)	1.7520(8)	1.018(13)	0.48	0.45
Square (Metropolis)	0.4423(13)	1.753(11)	1.3(2)	2.17	2.16
Triangular (Wolff)	0.27466(2)	1.7505(10)	1.005(13)	0.48	0.42
Triangular (Metropolis)	0.2752(8)	1.769(16)	1.11(19)	2.18	2.11
Hexagonal (Wolff)	0.65848(3)	1.7489(9)	0.998(13)	0.48	0.43
Hexagonal (Metropolis)	0.6580(9)	1.726(9)	0.94(11)	2.09	2.01

Tab. 1. Experimental results.

Geometry	χ^2_γ	χ^2_ν
Square (Wolff)	0.85	1.35
Square (Metropolis)	0.36	0.90
Triangular (Wolff)	1.18	0.73
Triangular (Metropolis)	1.04	0.79
Hexagonal (Wolff)	1.17	1.49
Hexagonal (Metropolis)	1.00	0.95

Tab. 2. Reduced χ^2 from the scaling fits to find γ and ν respectively.

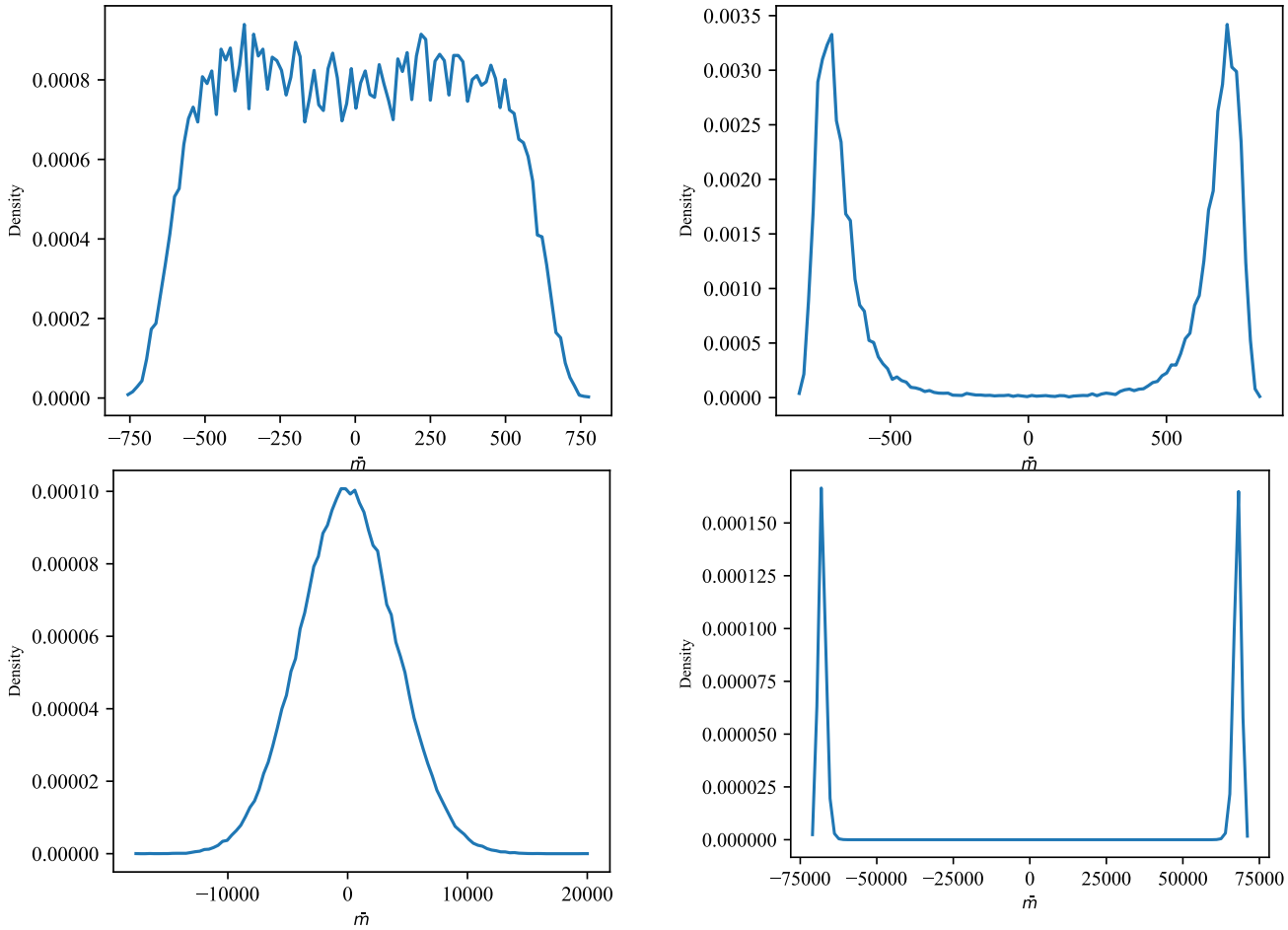

Fig. 13. Histograms of the sampled probability of total magnetization. The four pictures are related to a triangular lattice. The two at the top have been sampled with $L = 30$, the two at the bottom with $L = 300$. At left, $\beta = 0.261$ (above the critical temperature) and the right ones at $\beta = 0.282$ (in the broken phase).

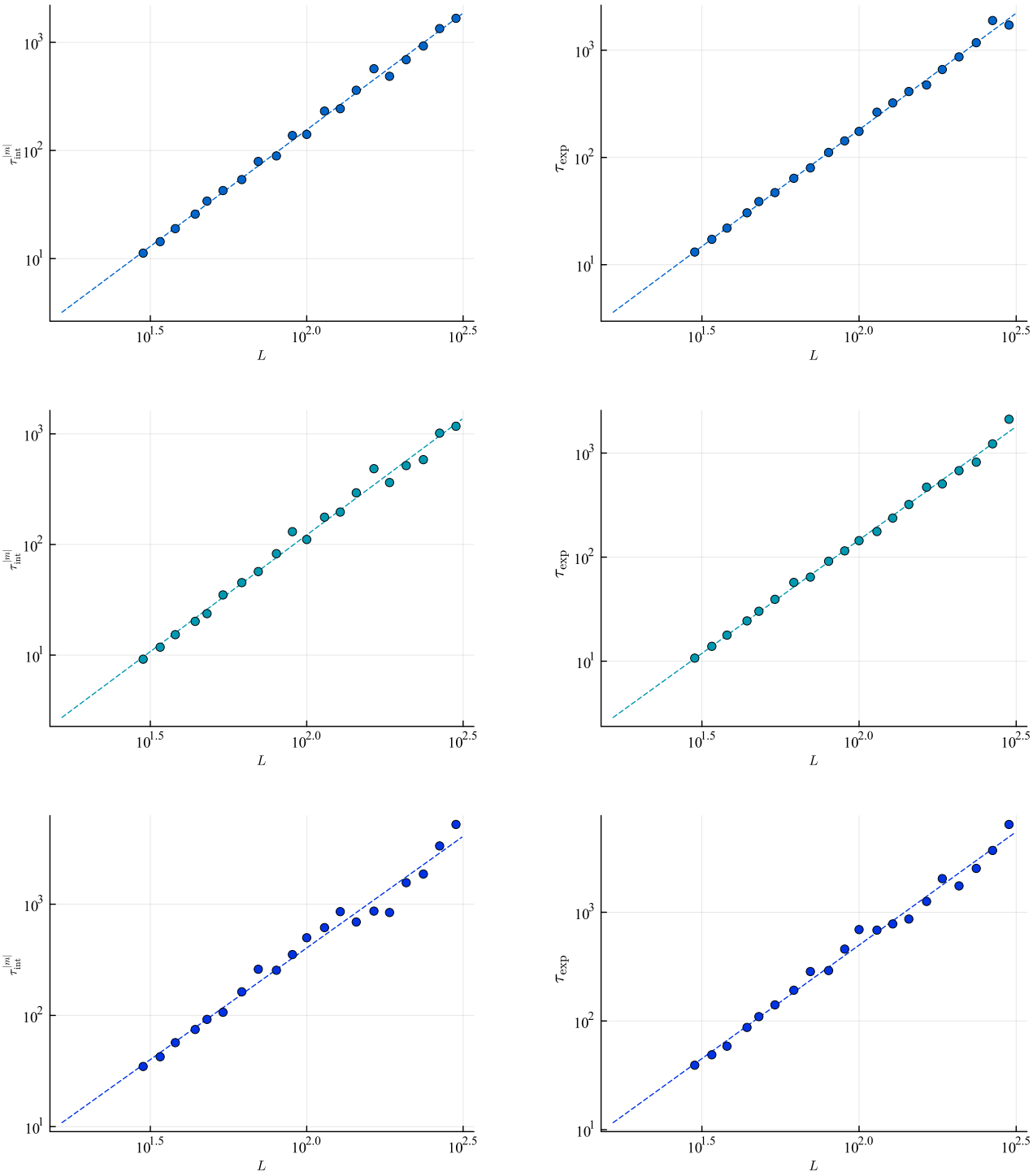

Fig. 14. Scaling of the magnetization τ_{int} (on the left column) and τ_{exp} of the MCMC (on the right one) for square, triangular, and hexagonal lattices, respectively, with Metropolis algorithm.

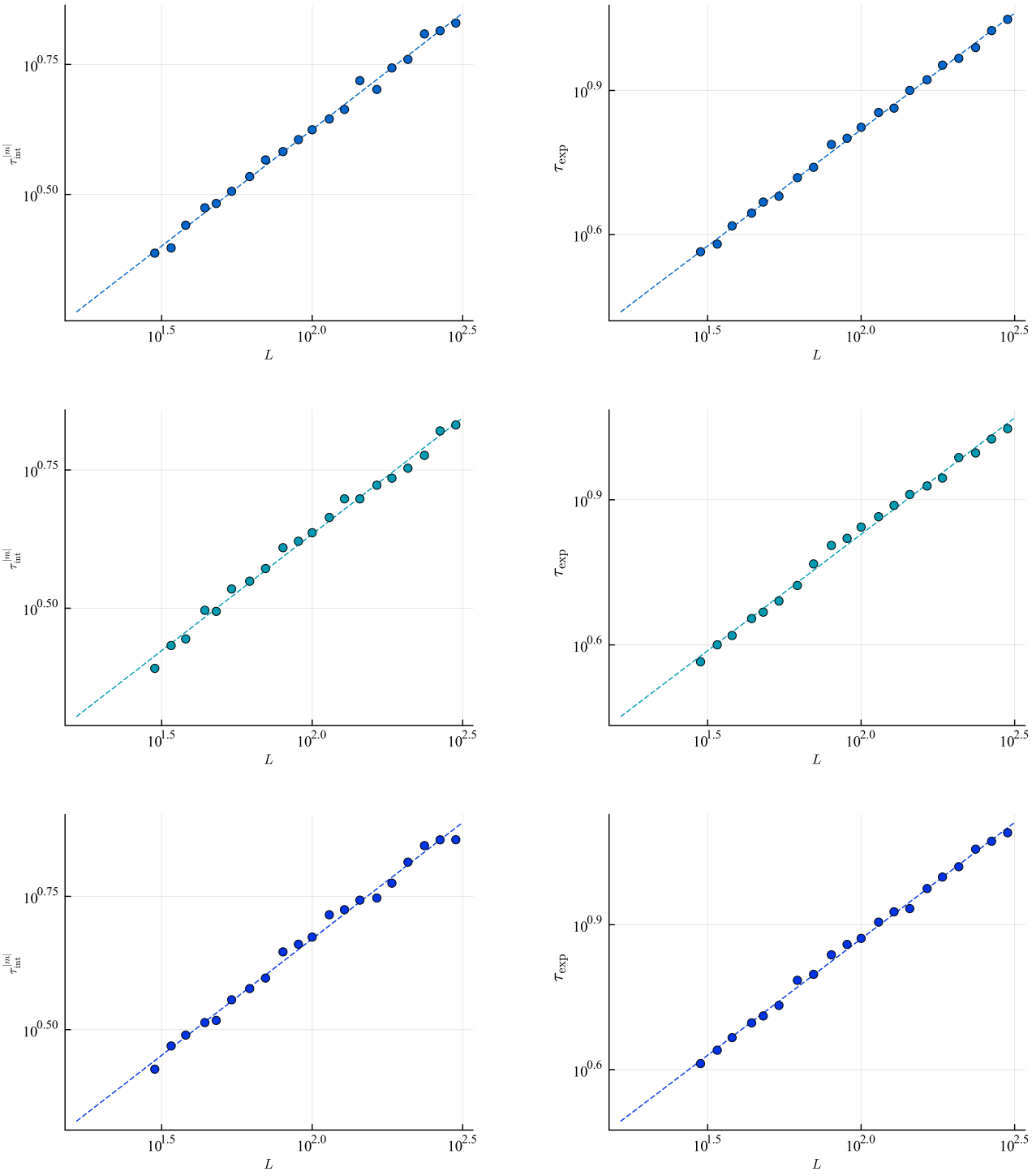

Fig. 15. Scaling of the magnetization τ_{int} (on the left column) and τ_{exp} of the MCMC (on the right one) for square, triangular, and hexagonal lattices, respectively, with Wolff algorithm.